

Structuration de documents

M1 MIAGE - S8
Groupe 1

Introduction.....	2
Conception.....	3
Structuration MongoDB.....	3
Collection des sources (G_FFST_sources).....	3
Collection des articles (G_FFST_articles).....	3
Choix de modélisation.....	4
Patterns.....	4
Indexation.....	4
Maquettes de l'application.....	5
Page de consultation.....	5
Page d'administration.....	7
Diagramme de navigation.....	7
Développement.....	8
Structuration de nos sources.....	8
Parser.....	8
Optimisation des requêtes.....	9
Situation initiale : index sur _id uniquement (index par défaut).....	9
Après ajout d'un index sur origin (origin: 1).....	9
Après ajout d'un second index sur publication_date (publication_date: 1).....	10
Conclusion et perspectives.....	10
Bilan de fonctionnalités.....	10
Ce qu'on a pu réaliser.....	10
Ce qu'on a pas pu réaliser.....	11
MongoDB.....	12
Pertinence de l'outil.....	12
Défis de l'outil.....	12
Points forts.....	12
Points faibles et pistes d'amélioration.....	12

Introduction

Dans le cadre de notre première année de master MIAE (Méthodes Informatiques Appliquées à la Gestion des Entreprises), nous avons eu l'opportunité de réaliser un projet visant à concevoir un système de génération d'un nuage de mots à partir de titres d'actualités. Ce projet s'inscrit dans le cadre du cours de Structuration de documents. Il nous a permis d'appliquer les notions théoriques abordées en cours tout en développant nos compétences en programmation, en travail d'équipe et en gestion de projet.

L'objectif de ce projet est de développer une application web permettant de collecter automatiquement des actualités à partir de fichiers sitemap de différents médias, puis de stocker ces informations dans une base de données NoSQL MongoDB. L'application permet de consulter les articles collectés et offre une fonctionnalité d'analyse des tendances actuelles qui génère un nuage de mots au format SVG basé sur les mots les plus fréquents contenu dans les titres des articles sur une période donnée.

L'application doit être organisée autour de deux modes principaux :

1. Le mode administration permet à l'utilisateur de gérer les sources d'actualités en s'abonnant à des sitemaps, en supprimant des abonnements existants et en définissant la fréquence de récupération des nouvelles informations publiées.
2. Le mode consultation permet d'explorer les articles collectés selon différents critères tels que la source, la date de publication, le texte ou les mots-clés. L'utilisateur peut également accéder directement aux articles en cliquant sur leur titre, ce qui ouvre l'article dans un nouvel onglet tout en enregistrant l'horodatage de la consultation.

Le projet a été développé en Python 3, avec Flask comme framework web et PyMongo pour l'accès à la base de données MongoDB. Le versionnement du code a été géré avec Git pour faciliter la collaboration au sein de l'équipe.

Conception

Le projet vise principalement à développer une application web pour l'analyse et la visualisation de données textuelles. Cette application sera entièrement basée en langage Python.

L'architecture technique s'appuiera sur l'API MongoDB pour la gestion et la persistance des données. Ce choix est justifié par la flexibilité de cette base de données NoSQL orientée documents pour les données textuelles non structurées ou semi-structurées, et par ses hautes performances en lecture, essentielles pour l'analyse.

L'interface utilisateur sera développée avec Flask, un micro-framework web Python léger, qui permettra de construire rapidement les routes pour l'interaction utilisateur et la restitution des résultats. Toute la logique métier, du traitement des données à la communication avec la base, sera implémentée en Python.

Le nuage de mots sera produit au format SVG pour assurer une qualité d'affichage optimale et une bonne adaptabilité. L'objectif est de minimiser le temps de latence entre la requête d'analyse et l'affichage du nuage SVG, un processus qui implique des calculs de fréquences de mots et un algorithme de placement spatial.

Structuration MongoDB

Collection des sources (G_FFST_sources)

Chaque document de cette collection représente un abonnement à un sitemap. Le schéma retenu est le suivant :

```
{ "name": "Le Figaro",  
  "url": "https://www.lefigaro.fr/sitemap_news.xml",  
  "last_update": 1715420399.0,  
  "time_interval": 3600 }
```

Le champ "name" est extrait automatiquement depuis la balise <news:name> du sitemap. Le champ "last_update" est un timestamp Unix, et "time_interval" est exprimé en secondes.

Collection des articles (G_FFST_articles)

Chaque document de cette collection représente un article d'actualité collecté. Le schéma est le suivant :

```
{ "loc": "https://www.lefigaro.fr/international/...",  
  "title": "Titre de l'article",  
  "origin": "Le Figaro",  
  "publication_date": "2026-05-11T08:30:00+00:00",  
  "image_loc": "https://i.f1g.fr/media/...",  
  "last_consultation": 17683579...  
}
```

Le champ loc (URL de l'article) sert de clé métier unique pour dédoubler les insertions via l'opération upsert. La date de publication est conservée sous forme de chaîne ISO 8601, ce qui permet les comparaisons lexicographiques et les tris directement en base.

Le champ "last_consultation" est mis à jour lors de chaque clic sur le titre de l'article depuis la page Liste. Il est absent des documents non encore consultés.

Choix de modélisation

Nous avons opté pour une modélisation en documents plats plutôt que pour une imbrication sources / articles. Ce choix est justifié par plusieurs raisons :

1. Les articles sont interrogés indépendamment des sources (filtres croisés par source, date, mots-clés).
2. L'imbrication sources/articles produirait des documents très volumineux et rendrait les requêtes sur les articles plus complexes.
3. La référence de source dans chaque article permet les jointures applicatives simples et les index sur le champ.

Patterns

Deux patterns principaux ont été appliqués :

1. Pattern de référence : les articles référencent leur source via le champ origin, plutôt que d'embarquer un sous-document source complet. Cela évite la redondance de données et permet de mettre à jour les métadonnées de la source sans modifier l'ensemble des articles.
2. Pattern Upsert : l'emploi de la commande "update_one" avec le paramètre "upsert=True" s'appuie sur le champ loc comme critère de filtrage. Ce mécanisme garantit que les articles déjà présents sont mis à jour au lieu de créer des doublons lors de la synchronisation du sitemap.

Indexation

Trois index simples de type ascendant sont créés au démarrage de l'application (via la fonction set_up_indexes) :

1. origin : Filtrage des articles par source
2. publication_date : Filtrage et tri chronologique des articles et du nuage de mots
3. last_consultation : Recherche et filtrage par date/heure de consultation

Le champ loc n'est pas indexé explicitement car MongoDB crée automatiquement un index sur le filtre d'upsert lorsqu'il est utilisé de façon répétée. L'index sur title n'a pas été ajouté car la recherche par mots-clés utilise une regex, dont les performances avec index restent limitées en MongoDB sans index de texte full-text.

Maquettes de l'application

L'interface est structurée autour d'une barre de navigation principale avec deux onglets : Consultation et Administration. Le mode consultation dispose lui-même de deux sous-onglets : Liste (articles filtrables) et Nuage (générateur du nuage de mot).

La page d'administration affiche un formulaire d'ajout de source (URL du sitemap, intervalle de mise à jour en heures), ainsi que la liste des sources abonnées avec leur dernière et prochaine mise à jour, et un bouton de désabonnement.

Page de consultation

Page contenant la liste des articles



La maquette illustre l'interface de consultation des articles. Elle est structurée en deux niveaux de navigation : une barre principale avec 'Consultation' (sélectionné) et 'Administration', et une barre secondaire avec 'Liste' (sélectionné) et 'Nuage'. Le contenu principal est centré et intitulé 'Filtres de recherche'. Il comprend quatre champs : 'Source:' avec un menu déroulant sur 'Toutes', 'De:' avec un champ de date 'jj/mm/aaaa' et un calendrier, 'À:' avec un champ de date 'jj/mm/aaaa' et un calendrier, et 'Mots-clés:' avec un champ de texte 'mmmm'. Un bouton 'Rechercher' est placé en dessous de ces champs.

Cet écran présente l'ensemble des actualités collectées sous forme de liste chronologique (du plus récent au plus ancien). Pour chaque article, l'utilisateur visualise le titre, la source d'origine, la date de publication et une image d'illustration. Des outils de filtrage permettent d'affiner l'affichage par source via un menu déroulant, par plage de dates ou par recherche textuelle dans les titres.

Page contenant le nuage de point

Consultation Administration

Liste Nuage

Paramètres du nuage

Source: Toutes

Nombre de mots: 50

De: jj/mm/aaaa

À: jj/mm/aaaa

Générer

Télécharger le nuage

Ce volet propose une visualisation analytique des tendances. Le système génère un nuage de mots au format SVG où la taille de chaque mot (de 10 à 50px) est proportionnelle à sa fréquence d'apparition dans les titres des articles filtrés. Les mots sont disposés en anneaux avec une palette de couleurs bleues, offrant une lecture rapide des sujets dominants sur une période donnée.

Page d'administration

Consultation

Administration

Ajouter une source

URL du sitemap

Intervalle de mise à jour

Unité



Heures

▼

Ajouter

Sources abonnées

Le Figaro

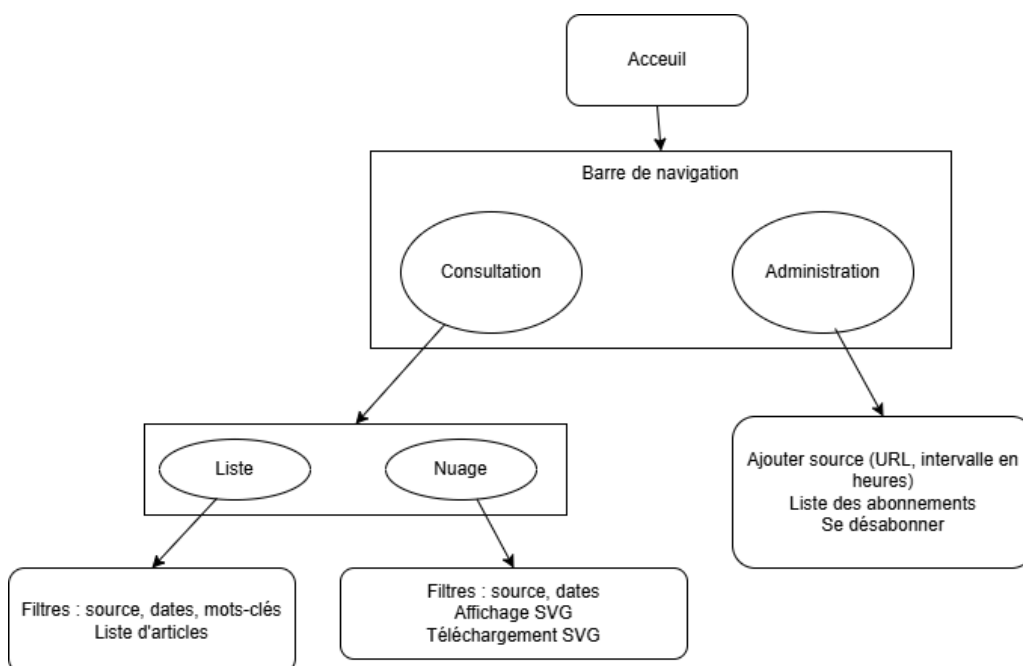
Dernière update: 08:59 11/05/26

Prochaine update: 09:59 11/05/26

Se désabonner

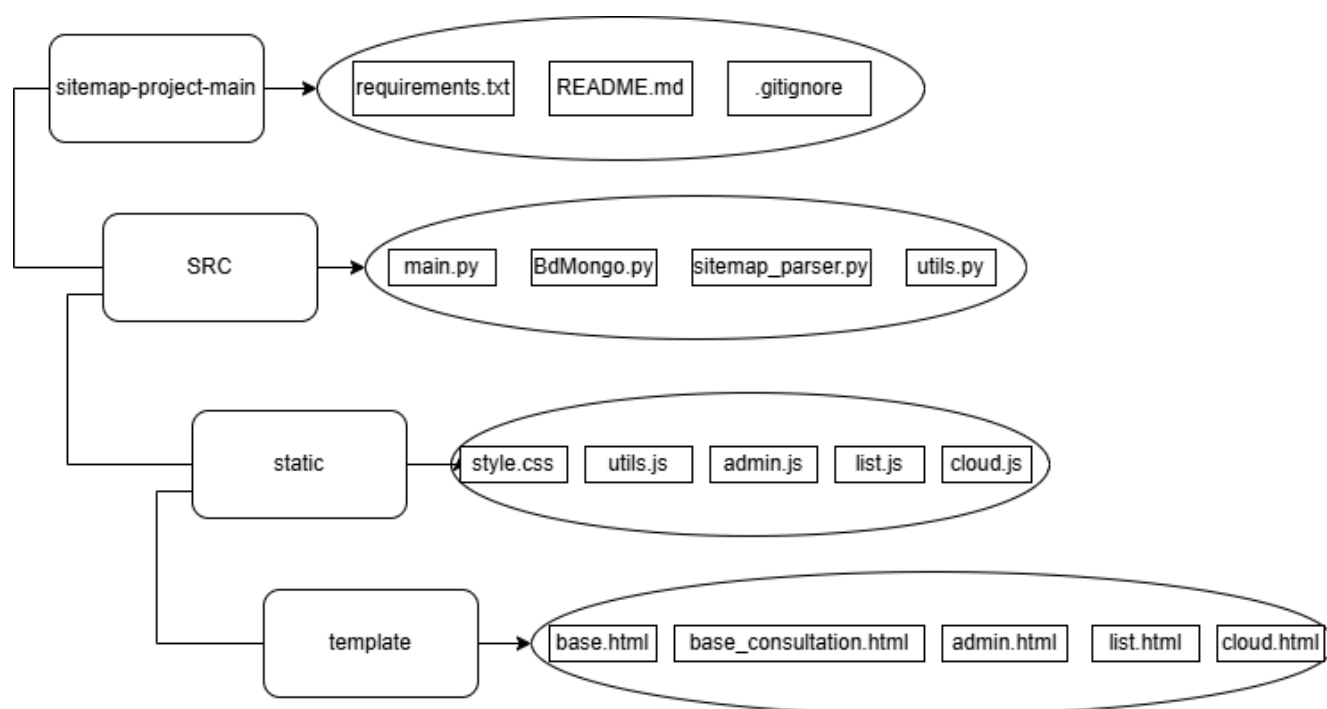
Destinée à la gestion des flux, cette page permet d'ajouter de nouvelles sources en saisissant l'URL d'un sitemap et l'intervalle de rafraîchissement souhaité. Elle affiche également un tableau de bord des abonnements actifs indiquant le nom de la source, l'horodatage de la dernière mise à jour et la planification de la prochaine collecte. Un bouton permet de supprimer une source, ce qui entraîne automatiquement la suppression de tous les articles qui y sont associés.

Diagramme de navigation



Développement

Structuration de nos sources



Parser

Le développement du parser de sitemap a suivi une démarche itérative en quatre étapes.

1. Avant d'écrire la moindre ligne de code, nous avons examiné manuellement plusieurs sitemaps pour identifier les balises présentes et comprendre leur organisation. Un sitemap est un document XML dont chaque élément `<url>` contient : la localisation de l'article (`<loc>`), des métadonnées de publication imbriquées dans `<news:news>` (titre, date, nom du média), et optionnellement une image (`<image:image>`). Plusieurs espaces de noms XML coexistent (namespace par défaut, `news:`, `image:`), ce qui nécessite une gestion explicite lors des requêtes XPath.
2. Nous avons retenu trois sources aux structures légèrement différentes pour valider la robustesse du parser : Le Monde (`lemonde.fr/sitemap_news.xml`), Le Figaro (`lefigaro.fr/sitemap_news.xml`) et Les Échos (`lesechos.fr/sitemap_news.xml`). Ce choix couvre différents gabarits et permet de détecter les variations de structure (présence ou absence de l'image, encodages différents).
3. Le parser (`sitemap_parser.py`) utilise la bibliothèque `lxml` pour le parsing XML.
 - La fonction `fetch()` récupère le contenu du sitemap via requests avec un User-Agent navigateur pour éviter les blocages.
 - La fonction `setup_namespaces()` extrait dynamiquement les espaces de noms déclarés à la racine du document et renomme le namespace par défaut en « `ns` » pour permettre les requêtes XPath.
 - La fonction `parse()` itère sur chaque élément `<url>` et extrait les champs `loc`, `title`, `publication_date`, `origin` et `image_loc` (optionnel).

- Une fonction séparée `get_source_name()` extrait uniquement le nom du média depuis la balise `<news:name>`, utilisée lors de l'abonnement à une nouvelle source.
4. Le parser a été testé en exécution directe sur les trois URL de test. Les résultats ont permis de valider l'extraction correcte de tous les champs, de détecter et corriger les cas où `image_loc` est absent, et de confirmer que l'upsert évite les doublons lors d'appels successifs.

Optimisation des requêtes

Une fois l'application terminée, nous sommes passés à l'optimisation des requêtes sur la collection `articles` de la base de données. Pour ce faire nous nous sommes principalement intéressés aux requêtes utilisant des filtres sur `origin` et `publication_date`, car ce sont les filtres qui sont communs à l'affichage de la liste et à la génération du nuage. Ainsi, nous avons pris cette requête de référence pour faire nos tests :

```
db.G_FFST_articles.find({
  "origin": "Libération",
  "publication_date": { "$gte": "2026-05-01", "$lte": "2026-05-06" }
}).explain("executionStats")
```

Situation initiale : index sur `_id` uniquement (index par défaut)

Avec une unique index sur `_id` (l'index par défaut), la requête obtient ces résultats :

```
nReturned: 57,
  executionTimeMillisEstimate: 1,
  works: 1178,
  advanced: 57,
  needTime: 1120,
  needYield: 0,
  saveState: 1,
  restoreState: 1,
  isEOF: 1,
  direction: 'forward',
  docsExamined: 1176
```

Après ajout d'un index sur `origin` (`origin: 1`)

Après ajout d'un index sur l'origine de l'article (`origin: 1`), beaucoup moins de documents ont été parcourus : on passe de 1176 à 358. L'index a donc bien optimisé la requête.

```
nReturned: 57,
  executionTimeMillisEstimate: 0,
  works: 359,
  advanced: 57,
  needTime: 301,
  needYield: 0,
  saveState: 0,
  restoreState: 0,
```

isEOF: 1,
docsExamined: 358,

Après ajout d'un second index sur publication_date (publication_date: 1)

L'ajout d'un index sur publication_date permet à MongoDB d'affiner encore davantage la sélection en ne parcourant que les documents dans la plage de dates demandée. Le nombre de documents examinés tombe à 61.

nReturned: 57,
executionTimeMillisEstimate: 0,
works: 63,
advanced: 57,
needTime: 4,
needYield: 0,
saveState: 0,
restoreState: 0,
isEOF: 1,
docsExamined: 61,
alreadyHasObj: 0,

En conclusion, l'ajout de ces deux index permet de réduire de 95 % le nombre de documents parcourus, passant de 1 176 à seulement 61. Ces résultats étant concluants, nous avons conservé ces deux index sur la collection articles. Ils sont créés automatiquement au démarrage de l'application via la fonction set_up_indexes() dans [BdMongo.py](#).

Conclusion et perspectives

Bilan de fonctionnalités

Ce qu'on a pu réaliser

Mode Administration

- Abonnement à un sitemap : saisie de l'URL et de l'intervalle de rafraîchissement (en heures). Le nom de la source est extrait automatiquement depuis la balise <news:name> du sitemap via la fonction get_source_name().
- Suppression d'un abonnement : suppression de la source et de tous les articles associés (delete_articles_from_source).
- Affichage de la date de dernière mise à jour et de la prochaine mise à jour prévue pour chaque source abonnée.
- Récupération automatique en arrière-plan : un planificateur APScheduler vérifie toutes les minutes si des sources doivent être mises à jour selon leur intervalle configuré.

Mode Consultation : Liste d'articles

- Affichage de tous les articles collectés, triés par date de publication décroissante

- Filtrage avec plusieurs critères : par source (menu déroulant), par plage de dates (date de début et date de fin), et par mots-clés (recherche regex dans le titre)
- Clic sur le titre d'un article : ouverture dans un nouvel onglet de l'article en question
- Affichage de la dernière date de consultation pour chaque article (si elle existe), directement dans la liste des articles

Mode Consultation : Nuage de mots

- Génération d'un nuage de mots SVG à partir des titres filtrés (source, plage de dates, nombre de mots n paramétrable).
- Algorithme de comptage de fréquences : tokenisation des titres (normalisation en minuscules, suppression des caractères spéciaux et des mots grammaticaux français courants), comptage via collections.Counter.
- Exclusion d'une liste prédéfinie de mots et caractère considérés comme inutile ou redondant

```
EXCLUDED_WORDS = ["e", ".", "près", "ou", "on", "plus", "moins", "...", "chez", "lors", "avant", "après",
"pendant", "est", "alors", "leur", "leurs", "sans", "en", "a", "-", "dans", "au", "aux", "il", "elle", "ils", "elles",
"nous", "vous", "tu", "je", "pas", "ne", "ma", "mon", "mes", "sur", "sous", "entre", "ses", "sa", "son", "se", "ce",
"cette", "cet", "quel", "quelle", "quels", "quelles", "quand", "que", "qui", "quoi", "dont", "comment", "pourquoi",
"on", "où", "là", "le", "la", "les", "un", "une", "de", "des", "du", "avec", "or", "pour", "et", "à", "avec", "par"]
EXCLUDED_CHARS = ["(", ")", "...", ".", ",", "!", "?", ":", ";", "%", "€", "$", "0", "1", "2", "3", "4", "5", "6", "7", "8",
"9", "<", ">", "n'", "n", "l'", "l", "qu'", "qu", "d'", "d", "j'", "j", "m'", "m"]
```

- Le placement des mots est organisé en anneaux concentriques autour du centre, en allant du mot le plus fréquent au moins fréquent. La taille de chaque anneau est calculée en fonction de la taille des mots qu'il contient. Étant donné que la largeur des mots est estimée et non mesurée avec exactitude, un très léger chevauchement peut survenir, notamment sur les mots très longs. Bien que ce système ne garantisse pas l'absence de superposition à 100 %, il la minimise de manière significative.
 - Rendu SVG : placement des mots en anneaux, taille proportionnelle à la fréquence (10 à 50px), couleurs dans une palette bleue.
 - Téléchargement du nuage généré au format SVG (via XMLSerializer côté client).
- Exemple du rendu du nuage de mots :



Ce qu'on a pas pu réaliser

- Filtrage des articles par catégorie de mots-clés (les keywords du sitemap ne sont pas extraits, seule la recherche textuelle dans le titre est supportée).

Par ailleurs, une limitation externe a été constatée : certains sitemaps (comme celui du Républicain Lorrain) ne sont pas mis à jour systématiquement par leurs éditeurs, indépendamment de la fréquence de rafraîchissement configurée dans l'application. Des articles récents peuvent donc être absents de la collecte sans que cela ne soit imputable à l'application.

MongoDB

Pertinence de l'outil

MongoDB s'est avéré être un excellent choix pour ce projet. Contrairement aux bases de données traditionnelles très rigides, MongoDB permet de stocker facilement des informations variées, comme des articles avec ou sans images, sans avoir à tout planifier à l'avance. Cela nous a aussi permis de mettre à jour les articles existants tout simplement tout en évitant les doublons.

Pour la création du nuage de mots, utiliser Python plutôt que les outils internes de la base de données s'est avéré être également un très bon choix. En effet, compter et nettoyer les mots des titres est une tâche complexe que Python gère beaucoup plus naturellement et avec plus de souplesse, ce qui facilite grandement le développement de cette fonctionnalité.

Défis de l'outil

1. Les sitemaps que nous lisons sont structurés de manière complexe, avec plusieurs niveaux d'information. Nous avons dû travailler pour que notre programme sache lire et interpréter correctement ces structures détaillées.
2. Tous les sites d'information ne respectent pas exactement les mêmes règles pour la création de leurs sitemaps. Nous avons dû ajuster notre outil pour qu'il puisse gérer des fichiers où des informations manquaient ou dont la structure était légèrement différente.
3. Nous pouvons configurer notre application pour vérifier les mises à jour très souvent, mais la fréquence réelle de mise à jour des articles dépend finalement des sites d'actualités eux-mêmes. Si un site ne met pas à jour son fichier d'index, nous ne pouvons pas récupérer les nouvelles informations, même si notre application tourne en arrière-plan.

Points forts

- Le code est séparé en parties distinctes pour gérer la base de données, lire les sources d'actualité et afficher les résultats. Cette structure facilite le développement et les modifications.
- La logique interne et l'interface utilisateur sont bien séparées, ce qui assure un fonctionnement rapide et fiable.
- La visualisation est créée sans avoir besoin de logiciels ou de bibliothèques externes complexes, ce qui la rend rapide à charger, légère et compatible avec tous les navigateurs.
- Un outil vérifie les sources toutes les minutes en arrière-plan. Cela permet de rafraîchir les données sans bloquer l'application ni ralentir les utilisateurs.

Points faibles et pistes d'amélioration

- La méthode actuelle est lente sur de grandes quantités de données. L'utilisation d'un outil de recherche spécialisé permettrait d'obtenir des résultats plus rapides et plus pertinents.

- Il n'y a pas de système de connexion par identifiant pour protéger la partie gestion, ce qui serait nécessaire pour une utilisation réelle hors du cadre scolaire.
- La liste des petits mots à ignorer (comme "le", "et",...) est fixe et limitée. Elle gagnerait à être gérée via un fichier externe ou un dictionnaire automatique plus complet.